Editor Scripts

Contents

- Files
- ShowIf
- EnemyEncounterManagerEditor
- SoundDataSOEditor
    - Writing a Custom Editor

Editor-only tools that keep Inspectors short and catch setup mistakes:

- **ShowIf:** hides Inspector fields that don't apply. Used across most of the game's components.
- **EnemyEncounterManagerEditor:** the Encounter Manager's custom Inspector.
- **SoundDataSOEditor:** the sound asset Inspector, with preview and warnings.

None of these affect the built game.

See also: Enemy Encounter & Spawning, Audio Manager, Enemy Behaviour.

Files

File	Folder
`ShowIfAttribute.cs`	**Not** an Editor folder (runtime scripts use it).
`ShowIfDrawer.cs`	**Editor** folder.
`EnemyEncounterManagerEditor.cs`	**Editor** folder.
`SoundDataSOEditor.cs`	**Editor** folder.

Getting the folders wrong breaks either the Inspector (attribute in an Editor folder) or the build (drawer outside one).

ShowIf

Hides a field unless another field has a certain value. Hidden fields disappear completely and come back the moment the condition changes. Hiding doesn't change the field's value.

```
        // Show when a bool is true
        [SerializeField] private bool damageOverTime;
        [ShowIf("damageOverTime")]
        [SerializeField] private float damageTickRate = 0.5f;

        // Show when a bool is false
        [ShowIf("alwaysAggro", false)]
        [SerializeField] private float aggroRange = 10f;

        // Show when an enum matches (cast to int)
        [ShowIf("enemyClass", (int)EnemyClass.Champion)]
        [SerializeField] private int numberOfPhases = 3;

        // Show when an object reference is assigned
        [ShowIf("damageFieldPrefab")]
        [SerializeField] private float damageFieldHeight = 1f;
```

Headers

A normal `[Header]` stays visible even when everything under it is hidden. Put the header on the first field of the group through `ShowIf` instead, and it hides with the field:

```
        [ShowIf("spawnDamageField", Header = "Ground Check")]
        [SerializeField] private LayerMask groundLayers = 1;
```

Rules

- Works inside `[Serializable]` classes and structs (and lists of them). The condition field is looked for next to the field first, then at the top level.
- Supported conditions: **bool**, **enum** and **object reference**. Anything else is always shown.
- A misspelled condition name means the field is always shown. Use `nameof(field)` to avoid typos.

Limits

- **One ShowIf per field.** Chaining doesn't combine conditions: in `GroundMovement`, Chase Range hides behind Never Give Up Chase, which hides behind Chase Enabled, but Chase Range still shows when Chase Enabled is off. For a true "A and B", use a custom editor.
- **On arrays and lists**, it hides each element but not the list itself (a Unity rule for property drawers).
- **Can't be combined with `[Range]` or `[Min]`.** Unity uses one drawer per field, so only one takes effect. `[Tooltip]` is fine.

> **In code:** `ShowIfDrawer` supports both Inspector systems. In UI Toolkit (Unity 6's default) it wraps the field in a container, hides it with `display: none`, and uses `TrackPropertyValue` on the condition so it updates instantly. In IMGUI it returns a height of 0 when hidden.

EnemyEncounterManagerEditor

Shows only the settings for the selected **Encounter Mode** (Standard or Arena), plus the right sub-settings for Arena's timing and location modes. See Enemy Encounter & Spawning for what they do.

When changing `EnemyEncounterManager`:

- **New fields don't appear** until they're added to the editor with `serializedObject.FindProperty("name")` and `EditorGUILayout.PropertyField`.
- **Renaming or removing a field breaks the Inspector**, because fields are found by name as a string. Update the editor in the same change.

SoundDataSOEditor

- **Play Preview / Stop** buttons. Preview picks a clip the same way the game does, at natural volume and pitch (for checking the clip, not the mix). Stops when the Inspector closes.
- **Warnings** for: no clips, inverted volume or pitch ranges, min distance above max distance, and a volume that works out to zero.
- Settings grouped into sections, with 3D distance settings only shown when Spatial Blend is above 0.

When changing `SoundDataSO`: new fields need adding to `OnEnable` and `OnInspectorGUI`. Fields are found with `nameof(SoundDataSO.field)`, so renames carry over automatically.

> **In code:** Unity has no public way to play audio in Edit Mode, so preview calls Unity's internal `UnityEditor.AudioUtil.PlayPreviewClip` and `StopAllPreviewClips` through reflection. If a Unity upgrade renames them, the buttons quietly stop working and nothing else breaks.

Writing a Custom Editor

1. Put it in an Editor folder and wrap it in `#if UNITY_EDITOR`.
2. Start `OnInspectorGUI` with `serializedObject.Update()` and end with `serializedObject.ApplyModifiedProperties()` (keeps undo and prefab overrides working).

3. Draw fields with `EditorGUILayout.PropertyField`, so tooltips and `ShowIf` still work.
4. Use `nameof(...)` to find properties where possible.

For most components `ShowIf` is enough. Use a custom editor when you need buttons, previews, warnings, or layout that depends on several fields.